Arquitectura de autenticación y control de roles

Matías Lira Breyan Sebastián

—

Desarrollo Web Profesional

—

José Miguel Carrera Pacheco

¿QUÉ ES AUTENTICACIÓN Y EN QUÉ SE DIFERENCIA DE AUTORIZACIÓN?

En los sistemas informáticos existen dos conceptos fundamentales para controlar el acceso: autenticación y autorización.

Aunque muchas veces se confunden, tienen funciones diferentes.

Autenticación

La autenticación es el proceso mediante el cual un sistema verifica la identidad de un usuario.

Es decir, el sistema comprueba que la persona que intenta ingresar es quien dice ser.

Ejemplos de autenticación:

- ingresar usuario y contraseña
- usar huella digital
- iniciar sesión con una cuenta de Google
- recibir un código de verificación

Autorización

La autorización ocurre después de la autenticación y consiste en determinar qué acciones puede realizar el usuario dentro del sistema.

Por ejemplo:

- ver información
- editar datos
- eliminar registros
- acceder a secciones administrativas

Diferencia principal

Concepto	Función
Autenticación	Verificar quién es el usuario
Autorización	Determinar qué puede hacer el usuario

Ejemplo práctico:

1. Un usuario inicia sesión con su contraseña → autenticación.
2. El sistema determina si puede acceder al panel de administrador → autorización.

¿QUÉ SON JWT, SESIONES Y OAUTH?

Existen diferentes métodos para gestionar la autenticación en aplicaciones web.

Sesiones

Las sesiones son un mecanismo tradicional de autenticación.

Cuando un usuario inicia sesión:

1. El servidor crea una sesión.
2. Se genera un identificador de sesión.
3. Ese identificador se guarda en una cookie del navegador.
4. En cada solicitud, el servidor verifica la sesión.

Las sesiones son comunes en aplicaciones desarrolladas con tecnologías como PHP, Java o .NET.

JWT (JSON Web Token)

Los JWT (JSON Web Tokens) son un tipo de token utilizado para autenticar usuarios en aplicaciones modernas.

Un JWT es un token firmado digitalmente que contiene información del usuario.

Ejemplo simplificado de un token:

header.payload.signature

Los JWT suelen utilizarse en aplicaciones que utilizan APIs o arquitecturas modernas.

OAuth

OAuth es un protocolo que permite que un usuario autorice a una aplicación a usar información de otra plataforma sin compartir su contraseña.

Este protocolo fue desarrollado por la organización Internet Engineering Task Force.

Ejemplo común:

- iniciar sesión con Google
- iniciar sesión con Facebook
- iniciar sesión con GitHub

La aplicación recibe un token de acceso en lugar de la contraseña del usuario.

¿CUÁNDO CONVIENE USAR TOKENS Y CUÁNDO SESIONES?

La elección depende del tipo de sistema que se esté desarrollando.

Cuándo usar sesiones

Las sesiones suelen ser adecuadas cuando:

- el sistema es monolítico
- el backend y frontend están en el mismo servidor
- se usan tecnologías tradicionales como PHP

Ventajas:

- fáciles de implementar
- control centralizado en el servidor

Cuándo usar tokens (JWT)

Los tokens son más adecuados cuando:

- se utilizan APIs REST
- hay aplicaciones móviles y web conectadas al mismo backend
- el sistema está basado en microservicios

Ventajas:

- permiten autenticación sin estado (stateless)
- funcionan bien en sistemas distribuidos

RIESGOS DE SEGURIDAD EN UN MAL DISEÑO DE AUTENTICACIÓN

Un sistema de autenticación mal diseñado puede generar graves problemas de seguridad.

Algunos riesgos comunes son los siguientes.

Robo de credenciales

Si las contraseñas no se almacenan de forma segura, pueden ser robadas en un ataque.

Por eso se recomienda usar hash y cifrado de contraseñas.

Ataques de fuerza bruta

Un atacante puede intentar muchas combinaciones de contraseñas hasta encontrar la correcta.

Para evitarlo se recomienda:

- limitar intentos de inicio de sesión
- implementar captcha
- usar autenticación multifactor

Secuestro de sesión

Ocurre cuando un atacante obtiene el identificador de sesión de un usuario y lo utiliza para acceder al sistema.

Tokens mal protegidos

Si los tokens se almacenan en lugares inseguros, pueden ser robados por scripts maliciosos.

Falta de control de permisos

Si el sistema no verifica correctamente los permisos, un usuario podría acceder a información que no le corresponde.

CÓMO SE DEFINEN ROLES Y PERMISOS EN SISTEMAS REALES

En sistemas reales se utilizan estructuras de roles y permisos para controlar el acceso a funcionalidades.

Este modelo se conoce como control de acceso basado en roles (RBAC).

Roles

Un **rol** es un conjunto de permisos asignado a un tipo de usuario.

Ejemplos de roles:

- administrador
- usuario normal
- supervisor
- invitado

Permisos

Los **permisos** son acciones específicas que un usuario puede realizar.

Ejemplos:

- crear registros
- editar información
- eliminar datos
- visualizar reportes

Ejemplo de estructura

Rol	Permisos
Administrador	Crear, editar, eliminar
Editor	Crear y editar
Usuario	Solo visualizar

Implementación en sistemas reales

En bases de datos suele implementarse con tablas como:

- **usuarios**
- **roles**
- **permisos**
- **usuario_rol**
- **rol_permiso**

Esto permite que el sistema controle de manera flexible qué puede hacer cada usuario.